

Foundations of Reinforcement Learning in the Context of Fine-Tuning and Distillation of LLMs for Machine Translation

Juan Antonio Pérez-Ortiz*, Universitat d'Alacant, Spain

Version 1.0, January 2026

Abstract

These notes present the mathematical foundations of reinforcement learning (RL) as applied to the fine-tuning and distillation of large language models (LLMs) for machine translation. It covers key concepts such as mathematical expectation, the log-derivative trick, importance sampling, policy gradients, and KL-regularized objectives. The aim is to provide a clear understanding of how these concepts are used in RL algorithms for LLMs. It is assumed that the reader has a basic understanding of probability theory, calculus, and machine learning, but no prior knowledge of RL is required. These notes are the result of a combination of human and AI-generated content, both working in tandem, and constitute the human author's understanding of the topic gained in the process of trying to comprehend the article "KDRL: Post-Training Reasoning LLMs via Unified Knowledge Distillation and Reinforcement Learning" by Xu et al. (2025).

1 Mathematical Expectation in the Context of LLMs

A large language model (LLM) produces a conditional probability distribution $\pi_\theta(o|q)$, where q is the input query (or prompt), o is the generated output, and θ are the model parameters. If we denote the sequence of tokens resulting from the model's output as $\hat{y}$, the probability of generating a specific output o given the input q is therefore expressed as:

$$P(\hat{y} = o|q) = \pi_\theta(o|q) \tag{1}$$

π_θ is often referred to as the *policy* in reinforcement learning (RL) terminology, but it is essentially the same as the conditional probability distribution produced by the LLM. Technically, a policy is a mapping from *states* (in this case, the input query q) to a probability distribution over *actions* (here, the possible outputs o).

The mathematical expectation operator $\mathbb{E}$ represents the weighted average or mean value of some function of the outcomes of the policy π_θ . For example, assume that we fix the input query of an LLM to $q = \text{Translate the following sentence into Spanish without providing any extra output at all: I see a man with a telescope, and, for simplicity, that our model is able to produce only two possible outputs with the following probabilities:}$

- o_1 : Veo un hombre con un telescopio with $\pi_\theta(o_1|q) = 0.7$
- o_2 : Veo un hombre que mira un telescopio with $\pi_\theta(o_2|q) = 0.3$

Now, consider that we compute a reference-free quality metric, such as COMET, for each output via a function $r(o, q)$ that extracts the source sentence to translate from the query q and uses the resulting

*Writing and learning done in tandem with AI tools (mostly, Gemini and Github Copilot).

input-output pair to compute the COMET score. Let's assume that this evaluation yields $r(o_1, q) = 0.9$ and $r(o_2, q) = 0.6$. We will call an evaluation function like this a *reward function* $r(o, q)$, as it provides feedback on the quality of the output o given the query q and constitutes a value that should be maximized during training. The expected reward (in our case, the expected COMET score) for this query is:

$$\mathbb{E}_{o \sim \pi_\theta(\cdot|q)}[r(o, q)] = \sum_{i=1}^2 \pi_\theta(o_i|q) r(o_i, q) = (0.7 \times 0.9) + (0.3 \times 0.6) = 0.81 \quad (2)$$

The reward (0.81) is much closer to the best possible reward (0.9) in this case because the model assigns a much higher probability to output o_1 . This illustrates the idea that around 70% of the times we will sample from the model, we will get the high-quality output o_1 and a reward of 0.9.

Note how we use the subscript $o \sim \pi_\theta(\cdot|q)$ to indicate that the output o is distributed according to the probability distribution π_θ given the query q . This notation is common in RL literature to clarify which distribution we are considering when computing the expectation as there could be multiple distributions involved in the same expression.

Random Variables and Expectation. The general definition of expectation involves a discrete random variable X that can take values from a set $\mathcal{X}$. The mathematical expectation, or expected value, is the weighted average of all possible outcomes:

$$\mathbb{E}_{x \sim P}[X] = \mathbb{E}[X] = \sum_{x \in \mathcal{X}} P(X = x) \cdot x = \sum_{x \in \mathcal{X}} P(x) \cdot x \quad (3)$$

where $P(X = x)$ is the probability of the random variable X taking the value x . We use the notation $x \sim P$ to indicate that x is distributed according to P , but we may omit it for brevity if the probability distribution is clear from the context, thereby using the simpler notation $\mathbb{E}[X]$. We may also abbreviate $P(X = x)$ as $P(x)$.

Note that the preceding equation implies that the random variable X takes values in a vector space (like $\mathbb{R}^n$) in which addition and scalar multiplication are defined. This excludes nominal (non-numerical) random variables such as a variable representing the output text of an LLM. To handle such cases we define a function like the reward $r(o, q)$ that maps the output o to a real number. This connects with the idea of the *law of the unconscious statistician* (or LOTUS), which states that for a random variable X and a function g of X , the expectation of $g(X)$ can be computed as:

$$\mathbb{E}[g(X)] = \sum_{x \in \mathcal{X}} P(x) \cdot g(x) \quad (4)$$

This means that we can compute the expectation of a function g of a random variable without explicitly knowing the distribution of g . In our case, the output o of the LLM is a random variable, and we can compute the expectation of the reward function $r(o, q)$ using the probabilities given by the model's policy $\pi_\theta(o|q)$ as we did earlier in equation (2) to obtain the expected reward of 0.81.

Monte Carlo Estimation of the Expectation. The set $\mathcal{X}$ of all possible values of a random variable can be very large or even infinite, making it impractical to compute the expectation in equation (3) by summing over all possible values. Luckily, the *Monte Carlo* method, developed in the 1940s by Stanislaw Ulam, John von Neumann and Nicolas Metropolis, provides a way to estimate the expectation by sampling from the distribution.

We may say that Monte Carlo method consists in *approximating the expected value with the mean of a computer-generated sample*. More formally, if we draw N independent samples $x_1, x_2, \dots, x_N$ from the distribution P , we can approximate the expectation in equation (3) as:

$$\mathbb{E}[X] \approx \frac{1}{N} \sum_{i=1}^N x_i \quad (5)$$

It can be shown that as N increases, this approximation converges to the true expectation according to the law of large numbers. As it turns out that many problems in lots of fields can be solved by computing

expectations, Monte Carlo methods provide a practical way to estimate these expectations when direct computation is infeasible. In particular, RL algorithms for LLMs often rely on expectations over the model’s policy, so Monte Carlo estimation becomes a crucial tool.

Following our previous example, if we sample 100 times from our previous LLM example, and we get, for example, output o_1 in 74 of those samples and output o_2 in 26 samples (recall that the true probabilities for those outputs were 0.7 and 0.3, respectively), we can estimate the expected reward as:

$$\mathbb{E}_{o \sim \pi_\theta(\cdot|q)}[r(o, q)] \approx \frac{1}{100} (74 \times 0.9 + 26 \times 0.6) = 0.822 \quad (6)$$

which approximates the exact expectation of 0.81 we computed earlier. Note that, in this case, the notation $o \sim \pi_\theta(\cdot|q)$ explicitly tells us how to generate our data samples for the Monte Carlo estimation: we must sample outputs o from the policy π_θ .

Obviously, our previous LLM example was oversimplified to illustrate the concept of expectation and Monte Carlo estimation. In practice, LLMs can generate a vast number of possible outputs for a given query, thereby making direct computation of expectations infeasible. The expectation in this case may be expressed as:

$$\mathbb{E}_{q \sim Q, o \sim \pi_\theta(\cdot|q)}[r(o, q)] = \sum_q Q(q) \sum_o \pi_\theta(o|q) r(o, q) \quad (7)$$

where $Q(q)$ is the probability of a potential query over the set of all possible queries. In practice, we will assume no prior knowledge about $Q(q)$ and will sample queries uniformly from a dataset in order to compute the average reward across those samples by Monte Carlo estimation. As the dataset will implicitly define the distribution Q , it will be treated as a constant during the optimization process. Therefore, in the following we will omit the sampling from Q and focus on the expectation over the model’s output distribution π_θ :

$$\mathbb{E}_{o \sim \pi_\theta(\cdot|q)}[r(o, q)] = \sum_o \pi_\theta(o|q) r(o, q) \quad (8)$$

Note that while we often omit the query q from the subscript to reduce notation overhead, we retain the output o to clarify which model’s distribution is currently being explored, as different LLMs, and consequently different policies, will be implied in our algorithms. As already discussed, computation of the expectation in the preceding equation is generally infeasible, and we will rely on Monte Carlo estimation by sampling outputs o from the model’s policy π_θ to approximate it. To alleviate notation overhead, we may also shorten the subscript further from $o \sim \pi_\theta(\cdot|q)$ to $o \sim \pi_\theta$ when the fact that there is a query q is clear from the context.

Laws of Convergence. Two important results underpin the reliability of Monte Carlo estimation methods. On the one hand, the *law of large numbers* states that as the number of samples N increases, the sample mean converges to the true expectation. This extends the idea expressed in equation (5) as follows:

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N x_i = \mathbb{E}[X] \quad (9)$$

On the other hand, the *central limit theorem* provides a measure of this approximation’s precision. It establishes that the estimation error decreases at a rate proportional to $1/\sqrt{N}$. This inverse square root relationship implies a law of diminishing returns: to reduce the current error by a factor of 10, the sample size must increase a hundredfold (from N to $100N$). Consequently, while Monte Carlo methods are guaranteed to converge, achieving high levels of precision requires a significantly larger computational effort as the number of samples grows. In practice, we will use Monte Carlo estimation in RL with the aim of obtaining reasonably good approximations of gradient expectations that can guide the optimization process, rather than seeking extremely precise estimates. The right direction is often more important than the exact magnitude of the gradient in optimization as other techniques, such as adaptive learning rates, can help adjust the step size during training.

Basic Properties of the Expectation. The expectation operator $\mathbb{E}$ satisfies several fundamental properties that are essential for manipulating objective functions as we will see later:

- Additivity: $\mathbb{E}[X_1 + X_2] = \mathbb{E}[X_1] + \mathbb{E}[X_2]$
- Linearity: $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$
- Expectation of a constant: $\mathbb{E}[c] = c$
- Product of independent variables: if X_1 and X_2 are independent, $\mathbb{E}[X_1 X_2] = \mathbb{E}[X_1] \mathbb{E}[X_2]$
- Linear combination: $\mathbb{E}[\sum a_i X_i] = \sum a_i \mathbb{E}[X_i]$

As we will see later, there is no simple property for the derivative of an expectation when the distribution itself depends on the parameters being optimized. Therefore, we cannot move the gradient operator ∇ inside the expectation without additional considerations. This means that in general:

$$\nabla_{\theta} \mathbb{E}_{o \sim \pi_{\theta}} [r(o, q)] \neq \mathbb{E}_{o \sim \pi_{\theta}} [\nabla_{\theta} r(o, q)] \quad (10)$$

2 The Log-Derivative Trick

In many optimization problems, we often need to calculate the gradient of an expectation. In particular, in the context of RL, there is an objective function $J(\theta)$ to be maximized, defined as the expected reward under the model's policy π_{θ} :

$$J(\theta) = \mathbb{E}_{o \sim \pi_{\theta}} [r(o, q)] = \sum_o \pi_{\theta}(o|q) r(o, q) \quad (11)$$

This equation means that we want to find the model parameters θ that maximize the expected reward $r(o, q)$ when sampling outputs o from the model's policy π_{θ} given the input query q so that we end up generating high-quality outputs according to the reward function. As usual, to optimize this objective using gradient-based methods, we compute the gradient of $J(\theta)$ with respect to the model parameters θ :

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{o \sim \pi_{\theta}} [r(o, q)] \quad (12)$$

$$= \nabla_{\theta} \sum_o \pi_{\theta}(o|q) r(o, q) \quad (13)$$

$$= \sum_o r(o, q) \nabla_{\theta} \pi_{\theta}(o|q) \quad (14)$$

where the property of the gradient of a sum as the sum of the gradients has been applied, and the reward $r(o, q)$ has been treated as a constant with respect to θ . The last expression has the important limitation that it is not an expectation anymore and, therefore, we cannot estimate it using Monte Carlo methods, which would leave us without a practical way to compute the gradient for optimization as summing over all possible outputs o is generally infeasible. Hopefully, we can apply the so called *log-derivative trick* to rewrite it in a form that allows us to express it as an expectation again.

The log-derivative trick is based on the chain rule expression for the derivative of a logarithm. Recall that for a differentiable function $g(x)$, we have:

$$\nabla \log(g(x)) = \frac{1}{g(x)} \nabla g(x) \quad (15)$$

We can rearrange this expression to obtain:

$$\nabla g(x) = g(x) \nabla \log(g(x)) \quad (16)$$

Applying this identity to equation (14), repeated here for clarity, we get:

$$\nabla_{\theta} J(\theta) = \sum_o r(o, q) \nabla_{\theta} \pi_{\theta}(o|q) \quad (17)$$

$$= \sum_o r(o, q) \pi_{\theta}(o|q) \nabla_{\theta} \log \pi_{\theta}(o|q) \quad (18)$$

$$= \sum_o \pi_{\theta}(o|q) r(o, q) \nabla_{\theta} \log \pi_{\theta}(o|q) \quad (19)$$

$$= \mathbb{E}_{o \sim \pi_{\theta}} [r(o, q) \nabla_{\theta} \log \pi_{\theta}(o|q)] \quad (20)$$

As it can be seen, we have successfully rewritten the gradient of the expectation in equation (12) as an expectation itself, which allows us to estimate it using Monte Carlo methods by sampling outputs o from the model’s policy π_θ , as already discussed. In the following, we will apply the log-derivative trick multiple times to get rid of expressions that are not directly amenable to Monte Carlo estimation.

Numerical Stability of the Log. Note that equation (20) could be further expanded by applying the chain rule again to the logarithm to give:

$$\nabla_\theta J(\theta) = \mathbb{E}_{o \sim \pi_\theta} \left[r(o, q) \frac{1}{\pi_\theta(o|q)} \nabla_\theta \pi_\theta(o|q) \right] \quad (21)$$

However, a couple of important numerical stability considerations motivate the use of equation (20) instead of the above expression:

1. Raw probabilities $\pi_\theta(o|q)$ can be extremely small. Dividing by these values during differentiation can lead to numerical instability, including NaNs. Log-probabilities reside in a much more stable range.
2. Models π_θ typically use a softmax output at the very last layer to normalize the logits and produce probabilities. Calculating the gradient of the log-softmax is computationally more efficient and stable than differentiating the raw probabilities.

Therefore, using the log-derivative trick not only allows us to express the gradient as an expectation but also enhances numerical stability during optimization.

3 Importance Sampling

Recall that the objective under RL is to maximize an expected reward as already discussed in equation (11), repeated here for clarity:

$$J(\theta) = \mathbb{E}_{o \sim \pi_\theta} [r(o, q)] = \sum_o \pi_\theta(o|q) r(o, q) \quad (22)$$

This equation implies that we will sample outputs o from the current model π_θ when estimating the expectation using Monte Carlo methods. However, in practice, we often sample from a fixed previous version of the model $\pi_{\theta'}$ (usually denoted as π_{old} in the RL literature) to avoid re-sampling after every parameter update. If we sampled from the very same model we are optimizing, we would have to discard our *rollouts*¹ after each training step as the distribution could have changed and we would no longer have valid samples. This could lead to extremely inefficient slow training. By sampling from a previous model $\pi_{\theta'}$, we can perform multiple optimization steps on the same set of samples and simply update $\theta' = \theta$ periodically to refresh the samples. This discrepancy between the sampling distribution and the current model being optimized requires us to adjust our expectation calculation accordingly, as we will see now.

In statistics, this is known as *importance sampling*. It shows us how to compute an expectation with respect to one distribution when we are actually sampling from another distribution. The process is also referred to as *change of measure*. The key idea is quite simple. If we multiply and divide by $\pi_{\theta'}(o)$ in the previous equation, we get:

$$\mathbb{E}_{o \sim \pi_\theta} [r(o, q)] = \sum_o \pi_\theta(o|q) \frac{\pi_{\theta'}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \quad (23)$$

$$= \sum_o \pi_{\theta'}(o|q) \frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \quad (24)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \right] \quad (25)$$

¹A *rollout* is a sequence of actions taken by the model in response to a given input, along with the resulting states and rewards. In the context of LLMs, a rollout could be the sequence of tokens generated in response to a prompt, along with any associated rewards or evaluations.

The previous steps are straightforward: we first multiply and divide by $\pi_{\theta'}(o)$, leaving the expression unchanged; then, we rearrange the terms to simply factor out $\pi_{\theta'}(o)$ and put it visually in front; the resulting expression can be easily recognized as an expectation over $\pi_{\theta'}$, as we are adding the values of the function $(\pi_{\theta}(o)/\pi_{\theta'}(o))r(o, q)$ over all possible outputs o weighted by their probabilities under $\pi_{\theta'}$. We may now estimate the expectation using Monte Carlo methods by sampling outputs o from the distribution $\pi_{\theta'}$.

The term $\pi_{\theta}(o)/\pi_{\theta'}(o)$ is known as the *importance weight* (*likelihood ratio* or *importance sampling weight* are also common terms), which may be denoted as $\rho_o(\theta)$. It corrects for the fact that we are sampling from a different distribution than the one we are optimizing. Intuitively, if a sample had a probability of 0.25 under $\pi_{\theta'}$ but has 0.5 under π_{θ} , it will appear half as often as it should because of the fact that we are sampling from $\pi_{\theta'}$. We therefore compensate by multiplying by an importance weight of $0.5/0.25 = 2$.

Note that for importance sampling to be a valid estimator, the support of π_{θ} must be contained within the support of $\pi_{\theta'}$. This condition ensures that the change of measure identity remains mathematically sound. If there is an output o where $\pi_{\theta}(o) > 0$ but $\pi_{\theta'}(o) = 0$, the importance weight cannot be defined for those points. More importantly, because the sampler will never visit these regions, their contribution to the expectation is lost. This leads to a *biased estimator* that systematically underestimates or overestimates the true value, as it fails to account for all relevant parts of the state space. The concept of biased estimators is discussed in more detail in section 8.

As already said, the primary reason for sampling from $\pi_{\theta'}$ via importance sampling is to avoid re-sampling tokens after every single gradient update, and therefore it is a practical consideration more than a theoretical one. But practically, importance sampling can introduce high variance in the gradient estimates, especially when the distributions π_{θ} and $\pi_{\theta'}$ differ significantly, as importance weights can become very large or very small. This high variance can lead to instability during training, making it difficult for the optimization process to converge. Techniques such as clipping the importance weights or updating $\theta' = \theta$ more frequently can help mitigate these issues. Let's discuss the former in more detail.

When training a model π_{θ} using samples from a previous model $\pi_{\theta'}$, we call this *off-policy* training, as opposed to *on-policy* training, where samples are drawn directly from the current model π_{θ} .

Clipping the Importance Weights. To mitigate the aforementioned high variance introduced by large importance weights, we can clip² them to a range $[1 - \epsilon, 1 + \epsilon]$ for some small $\epsilon > 0$. This clipping helps stabilize the training process by preventing any single sample from having an outsized influence on the gradient estimate. Clipping may be implemented in the objective function of equation (25) as follows:

$$\mathcal{J}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\text{clip} \left(\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)}, 1 - \epsilon, 1 + \epsilon \right) r(o, q) \right] \quad (26)$$

where the clipping operation is defined as:

$$\text{clip}(x, a, b) = \begin{cases} a & \text{if } x < a \\ x & \text{if } a \leq x \leq b \\ b & \text{if } x > b \end{cases} \quad (27)$$

When computing the gradient of this clipped objective $\nabla_{\theta} \mathcal{J}(\theta)$, if the importance weight falls outside the clipping range, the clip function becomes constant with respect to θ , resulting in a zero gradient contribution from that sample. This will result in the parameter updates ignoring those samples entirely.

Note that instead of using J to denote the objective function, we now use $\mathcal{J}$ to indicate that it is a *surrogate objective function*, meaning that it approximates the true objective J but is not exactly equal to it. A surrogate objective incorporates additional terms or modifications to facilitate optimization, such as clipping in this case or the KL-regularization term mentioned later.

Clipping with Advantage Functions. Some RL algorithms³ use an *advantage function* instead of the raw reward. The idea is to estimate how much better or worse an output is compared to some

²PyTorch implements this operation via the `torch.clamp` function.

³GRPO (Generalized Reinforcement Policy Optimization) is an example of such an algorithm.

Algorithm 1: Pseudo-code version of the surrogate objective of equation (29).

Input: advantage $\hat{A}(o, q)$, importance ratio $\rho_o(\theta) = \pi_\theta(o|q)/\pi_{\theta'}(o|q)$, clipping parameter ϵ
Output: Surrogate objective value

```

if  $\hat{A}(o, q) > 0$  then
  if  $\rho_o(\theta) > 1 + \epsilon$  then
    return  $(1 + \epsilon)\hat{A}(o, q)$ 
  end
end
else
  if  $\rho_o(\theta) < 1 - \epsilon$  then
    return  $(1 - \epsilon)\hat{A}(o, q)$ 
  end
end
return  $\rho_o(\theta) \hat{A}(o, q)$ 

```

baseline action. The advantage function $\hat{A}(o, q)$ can be defined as:

$$\hat{A}(o, q) = r(o, q) - B(q) \quad (28)$$

where $B(q)$ denotes the baseline function. In this case, equation (26) has a more elaborated version that combines both importance sampling and clipping with the advantage function:

$$\mathcal{J}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\min \left(\frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} \hat{A}(o, q), \text{clip} \left(\frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}(o, q) \right) \right] \quad (29)$$

The purpose of incorporating a baseline is to reduce the variance of the gradient estimates. By centering the reward around a baseline, we can focus on the relative quality of outputs rather than their absolute rewards. This often leads to more stable and efficient learning.

The rationale behind using the minimum of the unclipped and clipped terms may be more clear by considering an equivalent pseudo-code formulation that separates the cases of positive and negative advantages, as shown in Algorithm 1. When the advantage is positive, we want to prevent the importance weight from being too large as a result of $\pi_\theta(o|q)$ being much larger than $\pi_{\theta'}(o|q)$, so we clip the importance weight at $1 + \epsilon$. This avoids excessively rewarding that sample. Conversely, when the advantage is negative, we want to prevent the importance weight from being too small as a result of $\pi_\theta(o|q)$ being much smaller than $\pi_{\theta'}(o|q)$, so we clip the importance weight at $1 - \epsilon$. Note, however, that when the advantage is positive, and $\pi_\theta(o|q)$ is smaller than $\pi_{\theta'}(o|q)$ (which constitutes an undesirable situation, as the current model is assigning a probability lower than $\pi_{\theta'}$ to a better-than-baseline output), we do not apply any clipping and leave the gradient-based optimization to handle this case naturally, probably by increasing $\pi_\theta(o|q)$. Similarly, when the advantage is negative and $\pi_\theta(o|q)$ is larger than $\pi_{\theta'}(o|q)$ (which is also undesirable, as the current model is assigning a probability higher than $\pi_{\theta'}$ to a worse-than-baseline output), we do not apply any clipping either, thereby allowing the optimization process to decrease $\pi_\theta(o|q)$ as needed. Clipping, therefore, may be considered as a restriction over the rewarding, rather than a restriction over learning.

Figure 1 illustrates three different importance weights along a line, together with the clipping range for $\epsilon = 0.2$. In case the advantage is positive, only the square will be clipped to 1.2, while the circle and

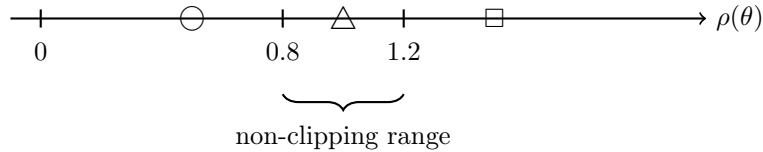


Figure 1: A line representing three importance weights $\rho_o(\theta) = \pi_\theta(o|q)/\pi_{\theta'}(o|q)$ at positions 0.5 (circle), 1.0 (triangle), and 1.5 (square). The segment between 0.8 and 1.2 represents the non-clipping range for $\epsilon = 0.2$. The main text discusses the different clipping cases for positive and negative advantages.

$\rho_o(\theta)$	$\hat{A}(o, q)$	a	b	$\min(a, b)$	Clipping?	Type
0.5 $\bigcirc$	+10	5	8	5	No	
1.0 $\triangle$	+10	10	10	10	No	
1.5 $\square$	+10	15	12	12	Yes	Upper
0.5 $\bigcirc$	-10	-5	-8	-8	Yes	Lower
1.0 $\triangle$	-10	-10	-10	-10	No	
1.5 $\square$	-10	-15	-12	-15	No	

Table 1: Examples of different cases for the surrogate objective of equation (29). The value of ϵ is assumed to be 0.2 in this example.

the triangle will remain unchanged. Conversely, in case the advantage is negative, only the circle will be clipped to 0.8, while the triangle and the square will remain unchanged.

To further illustrate it, Table 1 summarizes the different values for the importance weight, the advantage, and the corresponding unclipped (a) and clipped contributions (b), together with the resulting minimum value $\min(a, b)$ that will be used in the surrogate objective of equation (29). The last columns indicate whether clipping was applied and the type of clipping (upper or lower). As already discussed, clipping is only applied when the advantage and the importance weight are both pushing in the same direction (either both positive or both negative). In other cases, no clipping is applied.

Derivative of the Off-Policy Objective. The objective function defined in equation (25) can be differentiated as follows:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \right] \quad (30)$$

$$= \nabla_{\theta} \sum_o \pi_{\theta'}(o|q) \frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \quad (31)$$

$$= \sum_o \nabla_{\theta} \left(\pi_{\theta'}(o|q) \frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \right) \quad (32)$$

$$= \sum_o \pi_{\theta'}(o|q) \frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \quad (33)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \right] \quad (34)$$

where we have used the property that the gradient of a sum is the sum of the gradients, and treated $r(o, q)$ and $\pi_{\theta'}(o|q)$ as constants with respect to θ . Note that as the sampling distribution $\pi_{\theta'}$ does not depend on θ , we can move the gradient operator inside the expectation without further complications. Consequently, the log-derivative trick is not strictly necessary in this case. However, we may still apply the equation (16) of the log-derivative trick to express the gradient in terms of log-probabilities:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} r(o, q) \nabla_{\theta} \log \pi_{\theta}(o|q) \right] \quad (35)$$

As already discussed, using log-probabilities is often more numerically stable and computationally efficient. Notice how this expression resembles the on-policy gradient in equation (20), but with the expectation taken over $\pi_{\theta'}$ and the necessary inclusion of the importance weight. We could have arrived at the same result by changing the measure in equation (20) directly, which would have introduced the importance weight as already discussed in Section 3. Noticeably, the derivative is expressed as an expectation, allowing us to estimate it using Monte Carlo methods by sampling off-policy outputs from the distribution $\pi_{\theta'}$.

Finally, it should be emphasized that in equation (34), all terms are constant with respect to θ except for the numerator of the importance weight $\pi_{\theta}(o|q)$. Consequently, the computational graph for backpropagation only needs to track the operations involved in computing $\pi_{\theta}(o|q)$. At the risk of

stating the obvious, it is worth emphasizing that $\pi_\theta(o|q)$ represents the whole formulation of the model forward pass, including all layers and operations that lead to the computation of the output probabilities. By using this formulation is how the derivative with respect to each parameter in θ can be ultimately computed via the chain rule during backpropagation.

Expectation of the Importance Weight. For completeness and better understanding of importance sampling, we may also compute the expectation of the importance weight itself:

$$\mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} \right] = \sum_o \pi_{\theta'}(o|q) \frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} \quad (36)$$

$$= \sum_o \pi_{\theta'}(o|q) \quad (37)$$

$$= 1 \quad (38)$$

where we have used the definition of expectation, rearranged the terms, and finally applied the property that the probabilities of a distribution sum to 1. The fact that the expectation of the weight is 1 ensures the consistency of the change of measure, which allows the importance-sampled estimator to remain unbiased. If you multiply your sampled rewards by these weights, you are calculating the true expected reward under the target policy π_θ even though you are sampling from a different policy $\pi_{\theta'}$. Even though the average importance weight is 1, individual weights can vary significantly, which is why techniques like clipping or KL-regularization are often employed to control variance.

4 RL via the GRPO Algorithm

Group Relative Policy Optimization (GRPO) is an RL algorithm specifically designed to optimize LLMs efficiently by eliminating the need for a separate critic (value) model. Other algorithms like PPO (Proximal Policy Optimization) require a critic network to estimate the baseline B used in the advantage function of equation (28). This critic estimates the reward that can be expected from a given q and a partial output, and would typically be a copy of the LLM model itself with a different prediction head. The resulting set-up is often referred to as an *actor-critic* framework. Additionally, the Reinforcement Learning via Human Feedback (RLHF) approach also employs a separate neural model to provide the reward signal $r(o, q)$ based on human preferences. GRPO avoids the overhead of maintaining these additional large models by using a different strategy to compute the advantage function.

The key idea behind GRPO is to sample a group of outputs for each query and use the average reward of the group as the baseline B with an additional normalization so that the advantage has zero mean and unit variance within the group. The reward is computed via a *verifier* that assess the quality of each output. This verifier can be a smaller model or a heuristic function that is much less computationally intensive than a full critic network. The resulting framework is known as Reinforcement Learning via Verifiable Rewards (RLVR).

For each query q , GRPO samples a group of G independent outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta'}$. The reward for each output $r(o_i, q)$ is computed, and an advantage is estimated by normalizing the rewards within the group:

$$\hat{A}(o_i, q) \equiv \hat{A}_i = \frac{r(o_i, q) - \text{mean}(\{r(o_1, q), \dots, r(o_G, q)\})}{\text{std}(\{r(o_1, q), \dots, r(o_G, q)\})} \quad (39)$$

where $\text{mean}(\cdot)$ and $\text{std}(\cdot)$ denote the sample mean and standard deviation, respectively. This normalization ensures that the advantages have zero mean and unit variance within the group, which helps stabilize training. It also guarantees that some advantages will be positive (above average) and some negative (below average), providing a clear signal for the model to learn from. Negative advantages may be considered as wrong outputs and positive advantages as good outputs.

The surrogate objective for GRPO is then defined as:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\{o_i\} \sim \pi_{\theta'}} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta'}(o_i|q)} \hat{A}_i, \text{clip} \left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta'}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) \right) \right] \quad (40)$$

where the clipping mechanism is the same as already discussed.

5 Kullback-Leibler Regularization

Additionally, the summation in the GRPO objective in equation (40) may include a KL-regularization term to prevent the policy from deviating too much from the original distribution:

$$\mathcal{J}_{\text{GRPO-KL}}(\theta) = \mathbb{E}_{\{o_i\} \sim \pi_{\theta'}} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta'}(o_i|q)} \hat{A}_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta'}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) \right) - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right] \quad (41)$$

where $\beta > 0$ is a regularization coefficient, π_{ref} is a fixed reference model (often the pre-trained LLM before any RL fine-tuning), and $\text{KL}(\cdot \parallel \cdot)$ denotes the Kullback-Leibler divergence between two distributions. This term penalizes large deviations from the reference model, helping to maintain the quality of generated outputs. Note that π_{ref} is constant with respect to θ , so when computing the gradient of the KL term, only π_{θ} needs to be differentiated.

This regularization term is not unique to GRPO and can be applied to other RL algorithms as well. As we will see later, KL-regularization is particularly useful in the context of knowledge distillation via RL as a way to ensure that the student model does not diverge too much from the teacher model.

The KL divergence between two distributions π_a and π_b is defined as:

$$\text{KL}(\pi_a \parallel \pi_b) = \sum_x \pi_a(x) \log \frac{\pi_a(x)}{\pi_b(x)} \quad (42)$$

where the sum is taken over all possible values x of the random variable. The main properties of the KL divergence are:

- Non-negativity: $\text{KL}(\pi_a \parallel \pi_b) \geq 0$, with equality if and only if $\pi_a = \pi_b$.
- Asymmetry: $\text{KL}(\pi_a \parallel \pi_b) \neq \text{KL}(\pi_b \parallel \pi_a)$ in general; the later is known as the reverse KL divergence.

The application of RL for fine-tuning pre-trained LLMs leverages the high-quality prior distribution established during the initial pre-training phase. Unlike learning from scratch (like in other RL settings), where an agent must explore the state space without a baseline, fine-tuning starts from a highly capable model, which significantly enhances sample efficiency. Furthermore, the implementation of stability mechanisms (such as clipping and KL-regularization) effectively constrains the optimization process within a trust region. These methods prevent the policy from diverging excessively from other models, ensuring that the fine-tuning process remains stable while preserving the foundational linguistic intelligence of the original model.

6 RL for Machine Translation

The remaining sections discuss specific applications of RL to machine translation (MT) tasks as a way to further illustrate the concepts presented so far in a more concrete setting.

Given the formulation of RL via GRPO, its application to the fine-tuning of LLMs for machine translation (MT) tasks is straightforward. In this context, the model π_{θ} generates translations o_i for a given source sentence q . The reward function can be defined based on various MT quality metrics, such as BLEU, chrF, COMET, or MetricX. Depending on the specific metric used, the reward function may or may not require a reference translation t , or the source sentence itself. A more elaborated reward function may combine multiple metrics to capture different aspects of translation quality. For example, we could define:

$$r(o_i, q, t) = \alpha \cdot \text{BLEU}(o_i, t) + \beta \cdot \text{chrF}(o_i, t) + \gamma \cdot \text{COMET}(o_i, q) \quad (43)$$

where α , β , and γ are optional weights that balance the contribution of each metric. Note that COMET is used as a reference-free metric, so it does not require the reference translation t .

Instead of simply generating translations, the LLM can additionally be prompted to provide a reasoning before producing the final output. Following DeepSeek’s R1-Zero training method, the output can be instructed to follow this pattern:

```
<think> [model’s reasoning about the translation] </think>
<translate> [final translation] </translate>
```

In this case, the reward in equation (43) may include additional terms that evaluate that the format is followed correctly or the quality of the reasoning itself.

7 Knowledge Distillation of Translation Models via RL

Knowledge distillation (KD) is a technique used to transfer knowledge from a larger, more powerful *teacher* model to a smaller, more efficient *student* model. In the context of machine translation, KD can be employed to either train or fine-tune a student translation model by leveraging the outputs of a pre-trained teacher model, which may take the form of, for example, the tokens or the logits produced by the teacher when translating a set of source sentences. In addition to the traditional supervised learning approaches for KD, reinforcement learning can also be utilized to facilitate the distillation process.

The different approaches to knowledge distillation via RL can be classified based on two main criteria:

1. Whether the student model learns from samples generated by the teacher model (*off-policy*) or from its own generated samples (*on-policy*).
2. Whether the teacher model has to be kept in memory during training (*online*) or if its knowledge can be distilled into the student model without direct access to the teacher at training time (*offline*).

Off-policy methods suffer from the well-known problem of *exposure bias*, where the student model is trained on *golden* paths generated by the expert teacher, but during inference, it must generate its own tokens autoregressively. This discrepancy makes that during training the student never learns to recover from its own mistakes, leading to poor inference performance when it generates states (that is, sequence prefixes) that were not seen during training. On-policy methods mitigate this issue by allowing the student to learn from its own distribution, thereby exposing it to the types of errors it may make at inference time.

In this report, we will delve into two specific scenarios: off-policy distillation with an offline teacher, and on-policy distillation with an online teacher.

Off-Policy Distillation with an Offline Teacher. Under this setting, the student model learns from outputs generated by a fixed teacher model. The teacher’s outputs are pre-computed and stored in a dataset, which the student then uses for training. In mathematical terms, the approach is not very different from the one discussed in Section 6, with the main difference being that the reward function is designed to measure, via metrics such as BLEU, how closely the student’s outputs match those of the teacher instead of ground-truth references.

A common non-RL approach to KD known as *sequence-level KD* consists of maximizing the likelihood of the student’s outputs directly via supervised fine-tuning (SFT). This reduces to the well-known maximum likelihood estimation (MLE) framework, where the student model is trained to maximize the probability of the teacher’s outputs given the source sentences. In this scenario, the objective is to minimize the cross-entropy loss $\mathcal{L}_{\text{SFT}}(\theta)$ between the teacher’s output distribution and the student’s output distribution:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(q,o) \sim \mathcal{D}} [\log \pi_{\theta}(o|q)] = -\mathbb{E}_{(q,o) \sim \mathcal{D}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \log \pi_{\theta}(o_t|q, o_{<t}) \right] \quad (44)$$

where $\mathcal{D}$ is the dataset of source sentences q and corresponding outputs o generated by the teacher model. Note that in the previous equation, we have expressed the log-probability of the entire output sequence

o as the sum of the log-probabilities of each token o_t given the source sentence and the preceding tokens $o_{<t}$, following the autoregressive factorization of the model. We may simplify the notation by removing the input q so that the fact that the outputs come from the teacher model is more explicit:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{o \sim \pi_T} [\log \pi_\theta(o|q)] = -\mathbb{E}_{o \sim \pi_T} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \log \pi_\theta(o_t|q, o_{<t}) \right] \quad (45)$$

If instead of a minimization problem we want to express it as a maximization one, we may define the SFT objective to be maximized as the negative cross-entropy loss:

$$J_{\text{SFT}}(\theta) = -\mathcal{L}_{\text{SFT}}(\theta) = \mathbb{E}_{o \sim \pi_T} [\log \pi_\theta(o|q)] \quad (46)$$

The discussion of cross-entropy and SFT is out of the scope of this document, but, interestingly, minimizing the loss $\mathcal{L}_{\text{SFT}}(\theta)$ is equivalent to minimizing the *forward* KL divergence between the teacher and student models, a problem that can be approached via RL as we will prove next. In order to do so, first recall that, as already introduced in Section 5, the forward KL divergence between the teacher model π_T and the student model π_θ is defined as:

$$\text{KL}(\pi_T \parallel \pi_\theta) = \sum_o \pi_T(o|q) \log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \quad (47)$$

$$= \mathbb{E}_{o \sim \pi_T} \left[\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \right] \quad (48)$$

$$(49)$$

which can be approximated via Monte Carlo sampling by drawing outputs o from the teacher model π_T . By applying the property of logarithms $\log(a/b) = \log a - \log b$, we can rewrite the forward KL divergence as:

$$\text{KL}(\pi_T \parallel \pi_\theta) = \mathbb{E}_{o \sim \pi_T} [\log \pi_T(o|q) - \log \pi_\theta(o|q)] \quad (50)$$

$$= \mathbb{E}_{o \sim \pi_T} [\log \pi_T(o|q)] - \mathbb{E}_{o \sim \pi_T} [\log \pi_\theta(o|q)] \quad (51)$$

where we have applied the linearity of expectations (that is, the expectation of a sum is the sum of the expectations) to separate the two terms. Note that the first term $\mathbb{E}_{o \sim \pi_T} [\log \pi_T(o|q)]$ does not depend on the student's parameters θ , as it only involves the teacher model π_T , which is fixed. Therefore, when minimizing the forward KL divergence with respect to θ , this term can be treated as a constant and ignored. The second term, however, does depend on θ through the student model π_θ . Thus, minimizing the forward KL divergence corresponds to finding the parameters θ^* that satisfy:

$$\theta^* = \arg \min_{\theta} \text{KL}(\pi_T \parallel \pi_\theta) \quad (52)$$

$$= \arg \min_{\theta} (-\mathbb{E}_{o \sim \pi_T} [\log \pi_\theta(o|q)]) \quad (53)$$

$$= \arg \min_{\theta} \mathcal{L}_{\text{SFT}}(\theta) \quad (54)$$

$$= \arg \max_{\theta} J_{\text{SFT}}(\theta) \quad (55)$$

which is what we wanted to prove.

Let's now compute the derivative of the forward KL divergence with respect to the student's parameters θ just to illustrate the steps involved:

$$\nabla_{\theta} \text{KL}(\pi_T \parallel \pi_\theta) = \nabla_{\theta} \mathbb{E}_{o \sim \pi_T} \left[\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \right] \quad (56)$$

$$= \mathbb{E}_{o \sim \pi_T} \left[\nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \right) \right] \quad (57)$$

$$= \mathbb{E}_{o \sim \pi_T} \left[\underbrace{\nabla_{\theta} \log \pi_T(o|q)}_{\text{constant wrt } \theta} - \nabla_{\theta} \log \pi_\theta(o|q) \right] \quad (58)$$

$$= \mathbb{E}_{o \sim \pi_T} [0 - \nabla_{\theta} \log \pi_\theta(o|q)] \quad (59)$$

$$= \mathbb{E}_{o \sim \pi_T} [-\nabla_{\theta} \log \pi_\theta(o|q)] \quad (60)$$

$$= -\mathbb{E}_{o \sim \pi_T} [\nabla_{\theta} \log \pi_\theta(o|q)] \quad (61)$$

where we have used the fact that the teacher model π_T is fixed and does not depend on the student's parameters θ to enter the gradient operator inside the expectation. This is not the first time we see that under certain conditions the gradient operator can be moved inside the expectation: we already did it when deriving the off-policy objective in Section 3. To make the steps between equations (56) and (57) clearer, we start by expanding the expectation as a sum over all possible outputs:

$$\nabla_{\theta} \text{KL}(\pi_T \| \pi_{\theta}) = \nabla_{\theta} \mathbb{E}_{o \sim \pi_T} \left[\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] \quad (62)$$

$$= \nabla_{\theta} \sum_o \pi_T(o|q) \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \quad (63)$$

$$= \sum_o \nabla_{\theta} \left(\pi_T(o|q) \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \quad (64)$$

$$= \sum_o \pi_T(o|q) \nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \quad (65)$$

$$= \mathbb{E}_{o \sim \pi_T} \left[\nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \right] \quad (66)$$

Note again that in the RL context, we usually define objective functions to be maximized rather than minimized. Therefore, we may define our RL objective as the negative forward KL divergence:

$$J_{\text{FKL}}(\theta) = -\text{KL}(\pi_T \| \pi_{\theta}) = \mathbb{E}_{o \sim \pi_T} \left[\log \frac{\pi_{\theta}(o|q)}{\pi_T(o|q)} \right] \quad (67)$$

where we have rearranged the logarithm by applying $-\log(a/b) = -(\log a - \log b) = \log b - \log a = \log(b/a)$.

On-Policy Distillation with an Online Teacher. Consider the reverse KL divergence between the student model π_{θ} and the teacher model π_T , which is defined as:

$$\text{KL}(\pi_{\theta} \| \pi_T) = \sum_o \pi_{\theta}(o|q) \log \frac{\pi_{\theta}(o|q)}{\pi_T(o|q)} = \mathbb{E}_{o \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(o|q)}{\pi_T(o|q)} \right] \quad (68)$$

As it can be seen, the expectation is now taken over outputs sampled from the student model π_{θ} , which makes the training *on-policy*. We may define our objective function to maximize as:

$$J_{\text{RKL}}(\theta) = -\text{KL}(\pi_{\theta} \| \pi_T) = -\mathbb{E}_{o \sim \pi_{\theta}} \left[\log \frac{\pi_{\theta}(o|q)}{\pi_T(o|q)} \right] \quad (69)$$

$$= \mathbb{E}_{o \sim \pi_{\theta}} \left[-\log \frac{\pi_{\theta}(o|q)}{\pi_T(o|q)} \right] \quad (70)$$

$$= \mathbb{E}_{o \sim \pi_{\theta}} \left[\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] \quad (71)$$

where we have negated the reverse KL divergence to convert the minimization problem into a maximization one. Additionally, we have used again the property that:

$$-\log \frac{a}{b} = \log \frac{b}{a} \quad (72)$$

To optimize this objective J_{RKL} via gradient ascent, we need to compute its gradient with respect to the student's parameters θ . As already justified in Section 3 and in equation (25), sampling directly from π_{θ} implies that we have to resample after each parameter update, which is computationally expensive. Instead, we can use importance sampling to estimate the objective based on samples drawn from an older version of the student model, denoted as $\pi_{\theta'}$. Thus, we can rewrite the objective as:

$$J_{\text{RKL}}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] \quad (73)$$

We may now proceed to compute the gradient of this objective with respect to θ :

$$\nabla_{\theta} J_{\text{RKL}}(\theta) = \nabla_{\theta} \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] \quad (74)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\nabla_{\theta} \left(\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \right] \quad (75)$$

where we have moved the gradient operator inside the expectation since the sampling distribution $\pi_{\theta'}$ does not depend on θ . Applying the product rule for differentiation (that is, $\nabla(fg) = \nabla f g + f \nabla g$), we have:

$$\nabla_{\theta} J_{\text{RKL}}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} + \frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \right] \quad (76)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] + \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \right] \quad (77)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] + \xi \quad (78)$$

where we have separated the two terms resulting from the product rule using the linearity of the expectation operator. Let us look at the second expectation, which we have denoted as ξ , separately and consider it for a moment in order to show that it vanishes, that is, $\xi = 0$. Expressing the logarithm of the ratio as the difference of logarithms and using the property that the gradient of a sum is the sum of the gradients, we have:

$$\xi = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} \left(\log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right) \right] \quad (79)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} (\log \pi_T(o|q) - \log \pi_{\theta}(o|q)) \right] \quad (80)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \left(\underbrace{\nabla_{\theta} \log \pi_T(o|q)}_{\text{constant wrt } \theta} - \nabla_{\theta} \log \pi_{\theta}(o|q) \right) \right] \quad (81)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} (0 - \nabla_{\theta} \log \pi_{\theta}(o|q)) \right] \quad (82)$$

$$= -\mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} \log \pi_{\theta}(o|q) \right] \quad (83)$$

Expanding the expectation, we have:

$$\xi = - \sum_o \pi_{\theta'}(o|q) \cdot \frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \nabla_{\theta} \log \pi_{\theta}(o|q) \quad (84)$$

$$= - \sum_o \pi_{\theta}(o|q) \cdot \nabla_{\theta} \log \pi_{\theta}(o|q) \quad (85)$$

Notice how we have subtly performed a change of measure as the last equation is equivalent to the expectation $\mathbb{E}_{o \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(o|q)]$ over the current student model π_{θ} instead of the older version $\pi_{\theta'}$; although interesting on its own, we will not need to use this fact explicitly here. Continuing, we can further expand the last expression by deriving the logarithm via the chain rule:

$$\xi = - \sum_o \pi_{\theta}(o|q) \cdot \frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta}(o|q)} \quad (86)$$

$$= - \sum_o \nabla_{\theta} \pi_{\theta}(o|q) \quad (87)$$

$$= - \nabla_{\theta} \sum_o \pi_{\theta}(o|q) \quad (88)$$

$$= - \nabla_{\theta} 1 \quad (89)$$

$$= 0 \quad (90)$$

where we have used the fact that the sum of the probabilities over all possible outputs is equal to 1. Therefore, as we wanted to show, the second term resulting from the product rule vanishes ($\xi = 0$). Returning to the gradient expression in equation (77), we are left with:

$$\nabla_{\theta} J_{\text{RKL}}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\nabla_{\theta} \pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \right] \quad (91)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_{\theta}(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_{\theta}(o|q)} \nabla_{\theta} \log \pi_{\theta}(o|q) \right] \quad (92)$$

where in the last step we have applied the log-derivative trick from equation (16). This expression allows us to estimate the gradient using Monte Carlo methods by sampling outputs from the older student model $\pi_{\theta'}$. Note that this gradient expression resembles the policy gradient in equations (20) and (35), but with the reward function replaced by the log-ratio $\log(\pi_T/\pi_{\theta})$, and the expectation taken over $\pi_{\theta'}$ instead of π_{θ} which introduces in the equation the corresponding importance weight. Therefore, we can interpret the log-ratio as a type of reward signal that guides the student model to align its distribution with that of the teacher model in an on-policy manner. RL thus provides a principled framework to perform knowledge distillation while mitigating exposure bias. Intuitively, the student model is rewarded for generating outputs that the teacher model considers likely (high π_T compared to π_{θ}) while being penalized for outputs that the student itself considers likely (high π_{θ} compared to π_T), thereby encouraging the student to adjust its distribution towards that of the teacher.

The previous derivation has been explored so that we can familiarize ourselves with the mathematical manipulations involved in deriving RL objectives based on KL divergences.

Token-level Rewards. If instead of computing a single probability for the entire output sequence o , we express the model in an autoregressive form as a product of token-level probabilities, thereby allowing us to compute token-level rewards, we have:

$$J_{\text{RKL}}(\theta) = \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta'}(o_t|q, o_{<t})} \log \frac{\pi_T(o_t|q, o_{<t})}{\pi_{\theta}(o_t|q, o_{<t})} \right] \quad (93)$$

and

$$\nabla_{\theta} J_{\text{RKL}}(\theta) = \mathbb{E}_{o_t \sim \pi_{\theta'}(\cdot|q, o_{<t})} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta'}(o_t|q, o_{<t})} \log \frac{\pi_T(o_t|q, o_{<t})}{\pi_{\theta}(o_t|q, o_{<t})} \nabla_{\theta} \log \pi_{\theta}(o_t|q, o_{<t}) \right] \quad (94)$$

This token-level formulation allows for more granular feedback during training, as the student model can adjust its probabilities for individual tokens based on the teacher's guidance. As already said before, we keep the derivative of the logarithm in the expression for numerical stability and computational efficiency.

The reverse KL formulation including token-level rewards that allow us to arrive at the previous equations is the following one, which constitutes a more elaborated version of equation (68):

$$\text{KL}(\pi_{\theta} \parallel \pi_T) = \mathbb{E}_{o \sim \pi_{\theta}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \log \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_T(o_t|q, o_{<t})} \right] \quad (95)$$

Note how there is one single token log-probability ratio per time step t . This poses the limitation that the remaining tokens in the vocabulary are not considered. A more comprehensive approach would be to compute the KL divergence across the entire vocabulary at each time step, leading to the following formulation:

$$\text{KL}(\pi_{\theta} \parallel \pi_T) = \mathbb{E}_{o \sim \pi_{\theta}} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \sum_{v \in V} \pi_{\theta}(v|q, o_{<t}) \log \frac{\pi_{\theta}(v|q, o_{<t})}{\pi_T(v|q, o_{<t})} \right] \quad (96)$$

where V is the vocabulary of possible tokens. This full-vocabulary KL divergence captures the complete distributional differences between the student and teacher models at each time step, providing a significant reduction in gradient variance. By summing the exact probabilities and log-ratios of all possible tokens, the models receive a dense, deterministic supervision signal. This mitigates the instability inherent in policy gradients, leading to more precise updates, faster convergence, and reduced susceptibility

to reward hacking. Note that we do not average by dividing by the vocabulary size $|V|$ as each token in the vocabulary contributes to the total divergence proportionally to its probability under the student model π_θ .

In practice, however, computing the full-vocabulary KL divergence is often computationally prohibitive due to the large size of the vocabulary (often tens of thousands of tokens). Therefore, approximations such as sampling a subset of tokens or using top- k approximations are commonly employed to make the computation tractable. In this case, the top k tokens with the highest probabilities from the sampled policy are selected and a softmax is applied to renormalize the distribution before computing the KL divergence, thus obtaining a policy π_θ^k over the top k tokens only. The resulting KL divergence is then approximated as:

$$\text{KL}(\pi_\theta^k \| \pi_T) = \mathbb{E}_{o \sim \pi_\theta^k} \left[\frac{1}{|o|} \sum_{t=1}^{|o|} \sum_{v \in K} \pi_\theta^k(v|q, o_{<t}) \log \frac{\pi_\theta^k(v|q, o_{<t})}{\pi_T(v|q, o_{<t})} \right] \quad (97)$$

where $K \subseteq V$ is the set of top k tokens selected at each time step.

8 Statistical Considerations of KL Estimators in RL

In statistics, an *estimator* is a rule or formula that provides an estimate of a population parameter based on observed data. Estimators can be classified based on their statistical properties, such as bias and variance. An estimator is said to be *unbiased* if its expected value equals the true parameter being estimated. Conversely, an estimator is considered *biased* if its expected value deviates from the true parameter. *Variance* refers to the variability of the estimator’s estimates across different samples; an estimator with low variance produces estimates that are close to each other, while one with high variance yields estimates that can vary widely. A third important property is *consistency*, which means that as the sample size increases, the estimator converges in probability to the true parameter value.

The sample mean, for example, is an unbiased estimator of the population mean as its expected value equals the true mean of the population. The sample variance, on the other hand, is a biased estimator of the population variance, as its expected value tends to underestimate the true variance. If we divide the sum of squared deviations by $n - 1$ instead of n , we obtain an unbiased estimator known as Bessel’s correction.

Estimators are essential to Monte Carlo methods, which rely on random sampling to approximate true expectations. We aim to select estimators that balance bias and variance to achieve accurate and reliable estimates. In the context of RL, the choice of estimator can significantly impact the stability and efficiency of the learning process.

In our KD settings, the standard Monte Carlo estimator, often referred to as k_1 , provides an unbiased estimate of the divergence (as it targets the true KL divergence), but is characterized by high variance. This variance comes from the fact that it can be negative (specifically when $\pi_\theta(o|q) < \pi_T(o|q)$), while the true KL divergence is always non-negative. For the policies π_θ and π_T , the k_1 estimator is defined as:

$$k_1 = \log \frac{\pi_\theta(o|q)}{\pi_T(o|q)} \quad (98)$$

which corresponds to the estimator used in equation (68), repeated here for clarity:

$$\text{KL}(\pi_\theta \| \pi_T) = \mathbb{E}_{o \sim \pi_\theta} \left[\log \frac{\pi_\theta(o|q)}{\pi_T(o|q)} \right] = \mathbb{E}_{o \sim \pi_\theta} [k_1] \quad (99)$$

High variance in k_1 can lead to unstable training dynamics and the need for large batch sizes to achieve convergence. To mitigate training instability, researchers utilize alternative estimators that prioritize variance reduction, such as the improved estimators:

$$k_2 = \frac{1}{2} \left(\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \right)^2 \quad (100)$$

$$k_3 = \frac{\pi_T(o|q)}{\pi_\theta(o|q)} - 1 - \log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \quad (101)$$

The k_2 estimator is always non-negative and has a lower variance compared to k_1 , but it is biased in its scalar value as it performs a quadratic transformation of the log-ratio. However, it results in an unbiased gradient for the optimization objective, which is of primary importance in RL. In practice, using k_2 helps prevent training collapse by offering smoother optimization dynamics compared to k_1 . Furthermore, the k_3 estimator offers an unbiased estimate with lower variance by also ensuring that the value remains strictly positive.

Although we will not delve into the mathematical derivations of these estimators here, it is worth noting that the k_2 estimator is derived using a Taylor expansion of the KL divergence, while the k_3 estimator is what is known as a Bregman divergence. The fact that k_2 provides an unbiased gradient can be shown by differentiating it with respect to θ and demonstrating that its expected value matches the gradient of the reverse KL divergence:

$$\nabla_\theta \mathbb{E}_{o \sim \pi_\theta} [k_2] = \nabla_\theta \mathbb{E}_{o \sim \pi_\theta} \left[\frac{1}{2} \left(\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \right)^2 \right] \quad (102)$$

$$= \mathbb{E}_{o \sim \pi_\theta} \left[\log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \nabla_\theta \log \pi_\theta(o|q) \right] \quad (103)$$

$$= \mathbb{E}_{o \sim \pi_{\theta'}} \left[\frac{\pi_\theta(o|q)}{\pi_{\theta'}(o|q)} \log \frac{\pi_T(o|q)}{\pi_\theta(o|q)} \nabla_\theta \log \pi_\theta(o|q) \right] \quad (104)$$

$$= \nabla_\theta J_{\text{RKL}}(\theta) \quad (105)$$

which matches the gradient expression derived in equation (92) for the reverse KL objective $J_{\text{RKL}}(\theta)$ after a change of measure from π_θ to $\pi_{\theta'}$.

The webpage “Approximating KL Divergence” by John Schulman⁴ contains the code implementation of these estimators in PyTorch in Listing 1 that allows us to compare them empirically.

The code in Listing 1 estimates the KL divergence between two normal distributions p and q with different means using the three estimators k_1 , k_2 , and k_3 . It samples 100 million points from distribution q and computes the true KL divergence using PyTorch’s built-in function for reference. Then, it calculates the mean and standard deviation of the estimates from each estimator so that we may compare them with the true KL divergence. Three different mean values for distribution q are tested (0.1, 0.5, and 1.0) to observe how the estimators behave under different conditions.

```

1 import torch
2
3 import torch.distributions as dis
4
5 means = [0.1, 0.5, 1.0]
6
7 for mean in means:
8     print(f"\n=== Mean: {mean:.1f} ===")
9     p = dis.Normal(loc=0, scale=1)
10    q = dis.Normal(loc=mean, scale=1)
11    x = q.sample(sample_shape=(100_000_000,))
12    truekl = dis.kl_divergence(p, q)
13
14    print(f"True KL divergence (mean): {truekl.item():.8f}")
15
16    logr = p.log_prob(x) - q.log_prob(x)
17    k1 = -logr
18    k2 = logr ** 2 / 2
19    k3 = (logr.exp() - 1) - logr
20
21    estimators = {'k1': k1, 'k2': k2, 'k3': k3}
22
23    for name, k_tensor in estimators.items():
24        k_mean = k_tensor.mean().item()
25        k_variance = k_tensor.var().item()
26

```

⁴<http://joschu.net/blog/kl-approx.html>

```

27     print(f"{name} (mean): {k_mean:.8f}")
28     print(f"{name} (variance): {k_variance:.8f}")

```

Listing 1: Implementation of different KL divergence estimators in PyTorch.

The output of the code in Listing 1 is shown in Listing 2, where it is clearly seen how the k_1 estimator has a very high variance compared to the other two estimators, with k_3 being the one with the lowest variance among the three.

```

1  === Mean: 0.1 ===
2  True KL divergence (mean): 0.00500000
3  k1 (mean): 0.00499001
4  k1 (variance): 0.01000029
5  k2 (mean): 0.00501260
6  k2 (variance): 0.00005026
7  k3 (mean): 0.00500007
8  k3 (variance): 0.00005017
9
10 === Mean: 0.5 ===
11 True KL divergence (mean): 0.12500000
12 k1 (mean): 0.12501827
13 k1 (variance): 0.24996570
14 k2 (mean): 0.13279761
15 k2 (variance): 0.03514602
16 k3 (mean): 0.12498071
17 k3 (variance): 0.03400119
18
19 === Mean: 1.0 ===
20 True KL divergence (mean): 0.50000000
21 k1 (mean): 0.50010902
22 k1 (variance): 1.00001085
23 k2 (mean): 0.62505990
24 k2 (variance): 0.75011694
25 k3 (mean): 0.49999475
26 k3 (variance): 0.71799964

```

Listing 2: Output of the KL divergence estimators implementation in Listing 1.

For a comprehensive analysis of these estimators and their impact on post-training dynamics, refer to the document “Understanding KL divergence estimators in RL: from value approximation to gradient estimation” by Xihuai Wang.⁵

A Empirical Comparison of Forward and Reverse KL

Compared to the forward $\text{KL}(\pi_T \parallel \pi_\theta)$, minimizing the reverse $\text{KL}(\pi_\theta \parallel \pi_T)$ causes π_θ to seek the major modes of π_T and assign low probabilities to π_T ’s void regions, as illustrated in Figure 2, where a Gaussian mixture distribution is fit with a single Gaussian distribution using both forward KL and reverse KL. The graph in Figure 2 has been generated using the AI-generated Python code shown in Listing 3.

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import matplotlib.pyplot as plt
5  import numpy as np
6
7  class teacher_distribution:
8      def __init__(self):
9          # means and weights as requested (1, 4, 8) and (0.5, 1, 3.5)
10         self.means = torch.tensor([1.0, 4.0, 8.0])
11         self.weights = torch.tensor([0.5, 1.0, 3.5])
12         self.probs = self.weights / self.weights.sum()
13         self.s = 0.3 # sharper peaks to force the "gap" to zero

```

⁵<https://xihuai18.github.io/reinforcement-learning/2025/12/01/kl-estimators-en.html>

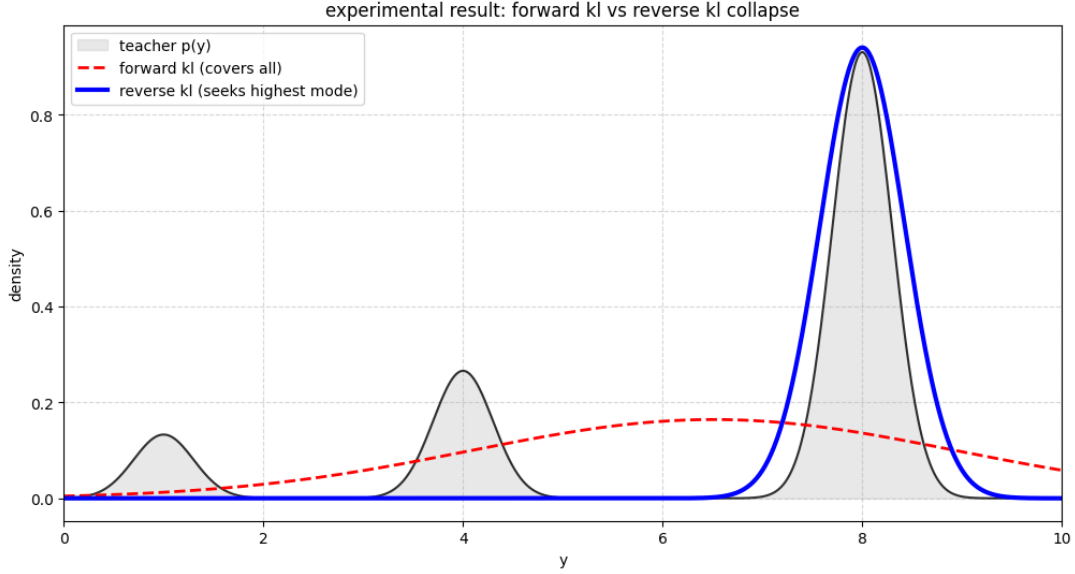


Figure 2: Empirical comparison of forward KL and reverse KL divergences. The gray curve represents the target distribution π_T , which is a mixture of three Gaussian distributions. The curves represent the approximating distribution π_θ , which is a single Gaussian distribution fitted using either forward KL (red) or reverse KL (blue) minimization. The forward KL fit covers all the modes of the target distribution, while the reverse KL fit focuses on the dominant mode and ignores the other, demonstrating mode-seeking behavior.

```

14
15 def sample(self, num_samples):
16     indices = torch.multinomial(self.probs, num_samples, replacement=True)
17     return torch.randn(num_samples) * self.s + self.means[indices]
18
19 def log_prob(self, x):
20     log_probs = []
21     for i in range(len(self.means)):
22         lp = -0.5 * ((x - self.means[i]) / self.s)**2 - 0.5 * np.log(2 * np.pi) - np
23     .log(self.s)
24     log_probs.append(torch.log(self.probs[i]) + lp)
25     return torch.logsumexp(torch.stack(log_probs), dim=0)
26
27 class student_model(nn.Module):
28     def __init__(self, init_mu=4.5, init_sigma=1.0):
29         super().__init__()
30         # starting with a sigma of 1.0 makes it local enough to feel the modes
31         self.mu = nn.Parameter(torch.tensor([init_mu]))
32         self.log_sigma = nn.Parameter(torch.tensor([np.log(init_sigma)]))
33
34     def get_std(self):
35         return torch.exp(self.log_sigma) + 1e-6
36
37     def sample(self, num_samples):
38         std = self.get_std()
39         # we use the reparameterization trick for the samples themselves
40         # but reinforce logic will treat them as discrete actions/samples
41         return torch.randn(num_samples) * std + self.mu
42
43     def log_prob(self, x):
44         std = self.get_std()
45         return -0.5 * ((x - self.mu) / std)**2 - self.log_sigma - 0.5 * np.log(2 * np.pi)
46
47 )
48
49 teacher = teacher_distribution()
50 # fwd student: classic mle behavior
51 fwd_student = student_model(init_mu=6.5, init_sigma=1.0)
52 # rev student: mini-llm rl behavior
53 rev_student = student_model(init_mu=6.5, init_sigma=1.0)
54
55 fwd_opt = optim.Adam(fwd_student.parameters(), lr=0.005)
56 rev_opt = optim.Adam(rev_student.parameters(), lr=0.005)

```

```

55 iterations = 3000
56 batch_size = 1024 # more samples reduce monte carlo noise
57
58 for i in range(iterations):
59     # --- forward kl (mode-covering) ---
60     fwd_opt.zero_grad()
61     x_p = teacher.sample(batch_size)
62     fwd_loss = -fwd_student.log_prob(x_p).mean()
63     fwd_loss.backward()
64     fwd_opt.step()
65
66     # --- reverse kl (mode-seeking via reinforce) ---
67     rev_opt.zero_grad()
68     x_q = rev_student.sample(batch_size).detach()
69
70     log_q = rev_student.log_prob(x_q)
71     log_p = teacher.log_prob(x_q)
72
73     # reward = log p - log q. minimizing kl(q||p) is maximizing e_q[log p - log q]
74     reward = log_p - log_q
75
76     # reinforce update: (reward - baseline) * grad(log_q)
77     baseline = reward.mean()
78     # we want to maximize reward, so we minimize negative reward
79     rev_loss = -((reward - baseline) * rev_student.log_prob(x_q)).mean()
80
81     rev_loss.backward()
82     rev_opt.step()
83
84 x = torch.linspace(0, 10, 1000)
85 p_pdf = torch.exp(teacher.log_prob(x)).numpy()
86 fwd_pdf = torch.exp(fwd_student.log_prob(x)).detach().numpy()
87 rev_pdf = torch.exp(rev_student.log_prob(x)).detach().numpy()
88
89 plt.figure(figsize=(12, 6))
90 plt.fill_between(x.numpy(), p_pdf, color='lightgray', alpha=0.5, label='teacher p(y)')
91 plt.plot(x.numpy(), p_pdf, 'k-', alpha=0.8)
92 plt.plot(x.numpy(), fwd_pdf, 'r--', lw=2, label='forward kl (covers all)')
93 plt.plot(x.numpy(), rev_pdf, 'b-', lw=3, label='reverse kl (seeks highest mode)')
94
95 plt.title('experimental result: forward kl vs reverse kl collapse')
96 plt.xlabel('y')
97 plt.ylabel('density')
98 plt.xlim(0, 10)
99 plt.legend()
100 plt.grid(True, which='both', linestyle='--', alpha=0.5)
101 plt.show()

```

Listing 3: Training script to generate Figure 2.